

02 linked list

Linked List Pattern

1. What is it, and when is it used?

A **linked list** is a linear data structure where elements ("nodes") each store a value and a pointer/reference to the next (and optionally previous) node. It's used whenever you need **sequential access with O(1) insertion/deletion** at known positions, without needing random access by index.

2. Why is it used?

- O(1) insertion/deletion at head/tail/middle (given a pointer to the node), vs O(n) shifting in an array.
- No need to pre-allocate a fixed size — grows/shrinks dynamically without resizing/copying like ArrayList.
- Natural fit for problems involving pointer manipulation, cycle structures, and merging ordered sequences.
- Underlying building block for other structures: hash map buckets (chaining), adjacency lists in graphs, LRU cache (doubly linked list + hashmap).

3. Where is it used?

- Implementing stacks/queues/deques
- LRU/LFU Cache (doubly linked list + HashMap)
- Polynomial representation, big number arithmetic
- Undo/redo history, browser back/forward navigation
- Music playlist "next/previous" navigation
- Graph adjacency lists
- Memory allocators (free lists)
- Blockchain (conceptually a linked list of blocks)

4. How do you identify it's a Linked List problem?

Signal phrases (2-minute test): - "Given the head of a linked list..." - "Reverse the list" / "reverse in groups of k" - "Detect a cycle" / "find where the cycle begins" - "Merge two/k sorted lists" - "Remove the Nth node from the end" - "Find the middle node" - "Flatten a multilevel linked list" - "Deep copy a list with random pointers" - Explicit `ListNode / Node { val, next }` class given in the problem statement

5. Can you identify it from constraints?

Constraint clue	Implication
"You may only traverse the list once"	Needs single-pass technique — often two pointers (fast/slow)
"O(1) extra space" required	Rules out storing nodes in a HashSet/array; must manipulate pointers in place
"Do not modify the list" but need info like middle/cycle	Fast & slow pointers (Floyd's) still O(1) space and non-destructive
Talks about "random pointer" alongside next pointer	HashMap-based deep copy or interweaving trick
n up to 10^4 - 10^5 , single list	Any O(n) or O(n log n) (merge sort on list) algorithm is fine

6. Types / Variants

1. **Singly Linked List** — node has only `next`.
2. **Doubly Linked List** — node has `next` and `prev` (O(1) deletion given the node, used in LRU cache).

3. **Circular Linked List** — tail points back to head.
4. **Skip List** — layered linked list for $O(\log n)$ search (used in Redis sorted sets).
5. Common sub-patterns:
 - **Fast & Slow Pointers (Floyd's Cycle Detection)** — find middle, detect cycle, find cycle start, detect palindrome list.
 - **Dummy Head Node** — simplifies edge cases when head itself might be removed/changed.
 - **In-place Reversal** — reverse whole list / reverse in groups of k / reverse between positions.
 - **Merge Technique** — merge two sorted lists (like merge step of merge sort), extend to merge k lists via heap or divide & conquer.
 - **Two-pointer with gap (N-th from end)** — advance one pointer N steps first, then move both together.

7. Rationale / Intuition

Because you can't jump to an arbitrary index in $O(1)$ (no random access), almost every linked-list technique is about **controlling one or more pointers cleverly to avoid a second pass or extra memory**: - *Fast/slow pointers* exploit the fact that if a faster pointer moving $2x$ speed ever laps a slower one, there must be a cycle — and the meeting point has a beautiful mathematical relationship to the cycle's start (Floyd's algorithm). - *Dummy head* removes the "is this the first node?" special case, so all deletions/insertions become uniform. - *In-place reversal* is really just re-pointing `next` links backward while walking forward, carrying `prev`, `curr`, `next` — the classic 3-pointer dance.

8. Time & Space Complexity

Operation	Time	Space
Traverse	$O(n)$	$O(1)$
Insert/Delete at known node	$O(1)$	$O(1)$
Search by value	$O(n)$	$O(1)$
Reverse (iterative)	$O(n)$	$O(1)$
Reverse (recursive)	$O(n)$	$O(n)$ call stack
Detect cycle (Floyd's)	$O(n)$	$O(1)$
Merge k sorted lists (heap)	$O(N \log k)$	$O(k)$
Merge sort on linked list	$O(n \log n)$	$O(\log n)$ recursion

9. Comparison with Other Patterns

Pattern	Random access	Insert/Delete (middle)	Extra space needed	Typical use
Linked List	$O(n)$	$O(1)$ given node pointer	$O(1)$ usually	Sequential ops, pointer surgery
Array/ArrayList	$O(1)$	$O(n)$ shift	$O(1)$	Index-based access, cache-friendly
Two Pointers (array)	$O(1)$	N/A	$O(1)$	Sorted array pair-sum, partitioning
Stack	$O(1)$ top only	$O(1)$ push/pop	$O(n)$	LIFO processing, can be built on linked list
Fast & Slow Pointer	—	—	$O(1)$	Cycle detection, middle element — a <i>technique</i> , often applied to linked lists

10. Reusable Java Templates

10.1 Node Definition

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}
```

10.2 Reverse a Linked List (Iterative)

```
class ReverseList {
    public ListNode reverse(ListNode head) {
        ListNode prev = null, curr = head;
        while (curr != null) {
            ListNode nextTemp = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nextTemp;
        }
        return prev; // new head
    }
}
```

10.3 Fast & Slow Pointers — Cycle Detection + Start of Cycle

```
class CycleDetection {
    public ListNode detectCycleStart(ListNode head) {
        ListNode slow = head, fast = head;
        boolean hasCycle = false;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) { hasCycle = true; break; }
        }
        if (!hasCycle) return null;

        slow = head;
        while (slow != fast) {
            slow = slow.next;
            fast = fast.next;
        }
        return slow; // start of the cycle
    }
}
```

10.4 Merge Two Sorted Lists (Dummy Head)

```
class MergeTwoLists {
    public ListNode merge(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;

        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
            else { tail.next = l2; l2 = l2.next; }
            tail = tail.next;
        }
        tail.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}
```

```
}
```

10.5 Remove Nth Node From End (Two Pointers with Gap)

```
class RemoveNthFromEnd {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode fast = dummy, slow = dummy;

        for (int i = 0; i < n; i++) fast = fast.next;
        while (fast.next != null) {
            fast = fast.next;
            slow = slow.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

11. Quick Recognition Checklist (2-minute test)

- ☐ Problem statement gives a `ListNode` / `Node` class with `next`?
- ☐ Asked to reverse, merge, detect cycle, or find middle/kth node?
- ☐ Constraint says $O(1)$ space with single traversal?
- ☐ Talks about “random pointer” or “multilevel” list?

If 2+ checked → Linked List (often combined with two-pointer/fast-slow technique).